cision Path:
  - If changes needed: Leave a comment requesting specific clarifications or modifications from the Issue Author/Assignee
    - Wait for response
    - Conduct any necessary follow-up discussions
    - Proceed to either Approval or Rejection (in rare cases)

  - If complete: Approve the issue, which automatically:
    - Publishes the CVE
    - Closes the issue

Timeline

Initial triage and first response: Within 2 days of receiving the publication request

CWEs, CVSS Scores, and Descriptions

1. Start with identifying an accurate CWE for the vulnerability
1. Review the CVSS score that the submitter provided

  - If the CVSS score is largely out-of-line with what you would expect based on the CWE and the description, confirm with the submitter that the score makes sense
  - If clear reasons exist for the unexpected CVSS metrics, add a note in the description to this effect. For example, "... Overall impact is limited due to the user only being able to affect their own account"
  - Note The CVSS score should make sense from an outside perspective when only having access to the CVE description and CWE

GPG Key

The email cve@gitlab.com has
a GPG key that the
Vulnerability Research team uses during CNA procesess.

Extending Public Key

The GitLab cve@gitlab.com email's public GPG key is set to expire every six months. Follow these steps to extend the expiration by another six months:

1. Have both the public and private keys to cve@gitlab.com locally.
1. List the keys with gpg --list-keys

```
console
$> gpg --list-keys
/home/user/.gnupg/pubring.kbx
------------------------------
pub   rsa4096 2020-00-00 [SC] [expires: 2020-06-00]
      AAAABBBBCCCCDDDDEEEEFFFFGGGGHHHHIIIIJJJJ
      uid          [ultimate] GitLab CVE <cve@gitlab.com>
```

sub   rsa4096 2020-00-00 [E] [expires: 2020-06-00]

1. Edit the key with gpg --edit-key AAAABBBBCCCCDDDDEEEEFFFFGGGGHHHHIIIIJJJJ
1. Set the expiration date on the key to another 6m with the expire interactive command:

```
console
$> gpg --edit-key AAAABBBBCCCCDDDDEEEEFFFFGGGGHHHHIIIIJJJJ
...
gpg> expire
...
Key is valid for? (0) 6m
```

Once the expiration on the public key is extended by another six months, export an armored version of the key with:

```
console
gpg --export --armor AAAABBBBCCCCDDDDEEEEFFFFGGGGHHHHIIIIJJJJ
```

You may also want to fetch the ID of the key with. The ID is the last 16 characters of the long-form key AAAABBBBCCCCDDDDEEEEFFFF[GGGGHHHHIIIIJJJJ]:

```
console
gpg --list-signatures cve@gitlab.com
```

Update Locations

The new public GPG key needs to be updated in the following locations:

- cves-workflow (internal)
- <https://about.gitlab.com/security/cve/>

SECTION: engineering/development/sec/security-risk-management/_index.md

Common Links

Slack channels:
  Main channel: ssrm
  Aggregated standup channel: ssrm-standup

Teams

Security Infrastructure

EM: {{< member-by-gitlab "ryaanwells" >}}

{{< member-and-role-by-gitlab "minac" "schmil.monderer" "bala.kumar" "ghavenga" "Quintasan" "srushik" "anarinesingh" >}}

Security Insights

EM: {{< member-by-gitlab "nmccorrison" >}}

{{< member-and-role-by-gitlab "bwill" "svedova" "charlieeekroon" "dpisek" "lorenzvanherwaarden" "wanderingperson" "sming-gitlab" "subashis" >}}

Security Platform Management

PM: {{< member-by-gitlab "smeadzinger" >}}

EM: {{< member-by-gitlab "or-gal" >}}

{{< member-and-role-by-gitlab "djadmin" "mamos-gl" "rossfuhrman" "ysiev" "ccharnolevsky" "gkatz1" "mfluharty" >}}

Security Policies

PM: {{< member-by-gitlab "g.hickman" >}}

EM: {{< member-by-gitlab "alan" >}}

{{< member-and-role-by-gitlab "aturinske" "sashikumar" "mcavoj" "Andyschoenen" "bauerdominic" "arfedoro" "mcrocha" >}}

SECTION: engineering/development/sec/security-risk-management/srm-planning.md

How we do planning

Our milestone planning is handled asynchronously as much as possible. Planning discussions are fluid and ongoing, in general they do not follow a predefined monthly schedule.

Planning Breakdown

Top priority features from upcoming release milestones go through Planning Breakdown with Product Managers (PMs), Product Designers (UX), Engineering Managers (EMs) & Engineers from the respective groups.
Weekly group-level synchronous meetings facilitate this discussion.
The list of issues to be discussed is provided by the PM at least 1 day prior to the meeting.
The expectation is that all attendees have reviewed the issues prior to the start of the meeting.
Attendees should add the carrot · emoji to signify that an issue has been reviewed in advance.

Questions to be answered:

1. Are requirements clear enough to understand intent of request?
2. Do we know the boundaries of work to be accomplished?

If the answer is "No" to either of these questions, discussion continues with the PM to improve the team's understanding of the request. If necessary, the discussion will continue asynchronously in the Epic or Issue and is brought back to a future weekly meeting.

If the answer is "Yes" to these questions the team estimates whether or not the issue can be delivered in a single iteration (ignoring any other work that may be in that same iteration). If it's determined that the issue under discussion cannot be delivered within a single
iteration, the team works with the PM to break it into multiple MVC Issues that can each be delivered in an iteration, are independent "slices" of value that can be used by a customer (so no
mocked UIs or backend-only work that is inaccessible), and when all delivered will completely fulfill the original issue's requirements.

EM output: Once all of the above requirements have been satisfied the EMs assign a frontend and backend engineer as respective DRIs to create Implementation Issues under the MVC epic(s). The Design issue created by UX is also closed at this point by the EM.

Engineering output: Frontend and backend DRIs create implementation issues following the Implementation template available for the gitlab-org project issues. Once they are done, they unassign themself and move issues to the workflow::refinement state.

Refinement

Issues in the workflow::refinement state are either assigned by EMs to individuals engineers for refinement.
Engineers assigned to refine issues are encouraged to ask questions and push back on PM if issues lack the information and/or designs required for successful refinement and execution.

We assign issues for refinement to ensure we have focus on the highest-priority items, as determined by Product Management. This is not an assignment to work on the issue.

Engineering output: Move issue into the workflow::ready for dev state and unassign themselves if they have completed refinement. Leave issue in workflow::refinement and assign the issue to their EM if for any reason refinement could not be completed. Confirm the issue has the appropriate work type classification.

Release Scope final & kickoff

By the week prior to the completion of the current milestone, the scope of the next release is finalized by EMs and PMs.

EM output: Deliverable labels are applied to issues we are committing to deliver. It's up to the EM's discretion what issues receive this label which is used in the calculation of our Say Do Ratio. Factors include: confidence that the issue will be completed in the milestone, completion of issues rolled over from the previous milestones, commitments with other groups or stakeholders.

EM output: Move issues that we are unlikely to deliver to the next iteration.
PM output: Issues with Deliverable labels in the workflow::ready for dev state have been confirmed to be in the correct priority order.

Refinement Guidelines

Backlog refinement is the most important step to ensure an issue is ready to move into development
and that the issue will match everyone's expectations when the work is delivered.

The goal of the refinement process is to ensure an issue is ready to be worked on by doing this:

Identify and resolve outstanding questions or discussions.
Identify missing dependencies (e.g. backend API).
Raise any questions, concerns or alternative approaches.
Outline an implementation plan.
Assign a weight to the issue.

Refinement steps for Engineers

1. Issues you need to refine will be assigned to you by your EM. Note the differences for bugs and spikes.
1. Backend/Frontend labels:
   If a backend engineer is required for the issue, ensure a backend label. Otherwise, remove any backend label, assign any relevant labels and you are done.
   If a frontend engineer is required for the issue, ensure a frontend label. Otherwise, remove any frontend label, assign any relevant labels and you are done.
1. Check the issue for completeness.
   Does it have the necessary designs?
   Is the functionality clearly articulated and is there a consensus or decision on how it should function?
   Are the technical details outlined?
   Has a consensus been reached or decision been made in areas of discussion?
   Are there dependencies? Call those out.
1. If the issue is not complete:
   Tag the relevant people that can help complete the issue and outline what is needed. Tag the appropriate EM and PM, so they know that the item can not be fully refined.
   If you are unable to resolve blockers to your refinement within a reasonable amount of time (2-3 days dependign on size of initative) see Failing Refinement.
1. Ensure the issue is fully understood.
   Update the issue description with the final description of what will be implemented.
   Update the issue description with an implementation plan.
   Update the issue description with verification steps.
   Ensure the issue title is accurate for the work being done.
   Open up new issues for 'follow-up' work, or work that was forced out of scope.
1. Assign a weight.

If the issue requires both frontend and backend work, it should be split and weighed independently.
1. Determine if a feature flag is needed.
   If you think that we should use the feature flag for a given issue, add ~"feature flag" label
    and add in the description a section called Feature Flag with the proposed name.
   Create a feature flag rollout issue to track the multiple stages of releasing with a feature flag.
   Consider creating a feature flag clean up issue if the removal of the feature flag will occur in a subsequent milestone.
1. Encourage Community Contributions.
   If the scope of the issue is well defined and there are no dependencies, consider adding
    contribution Labels.
   The ~"quick win" label is particularly helpful but note that you would
    be volunteering to mentor new contributors.
1. Refinement Review.
   If the weight you've assigned to the issue is 3 or less, move the issue directly to ~"workflow::ready for development".
   If the weight of the issue is greater than 3, unassign the issue from yourself and request a review from another engineer.
   When the reviewer agrees with the implementation plan and weight, they will unassign themself
    and move the issue to ~"workflow::ready for development".

Anyone should be able to read a refined issue's description and understand what is being solved, how it is solving the problem, and the technical plan for implementing the issue.

In order for someone to understand the issue and its implementation, they should not have to read through all the comments. The important bits should be captured in the description as the single source of truth.

Bug Diagnosis

Note the following differences when refining bugs:

1. As a guideline, spend no more than 1 hour per issue. Bugs that take too long to refine are indicative of a more complex issue.
1. Do not add weight. Our velocity represents the capacity to deliver new, bug-free features.
1. When you hit the time limit for refinement, it's ok to have uncertainty in the Implementation Plan. It's sufficient just to direct where you expect the code change to be (high or low level).

Refinement for Spikes

1. Do not add weights[^3].
1. Time-box how much time to spend on the issue.
1. The deliverable is typically an answer or solution to be used in upcoming issues.

[^3]: a spike doesn't directly add value to users so it shouldn't contribute to our velocity. The

information delivered by a spike is what will be useful to deliver direct value to users.

Refinement for Security Issues

The Security Developer process
can be daunting for first-timers. As part of refinement, ask for a volunteer to act as a "Security
Issue Release Buddy".

Failing Refinement

An issue should fail refinement if it can not be worked on without additional information or decisions to be made. To fail an issue:

1. Leave a comment on the issue that it can not be worked on, and highlights what still needs to
   be done.
2. Unassign yourself if you can not contribute further to issue at the current time.
3. Assign the workflow::blocked label.

Weights

Weights are used as a rough order of magnitude to help signal to the rest of the team how much work is involved. Weights should be considered an artifact of the refinement process, not the purpose of the refinement process.

It is perfectly acceptable if items take longer than the initial weight. We do not want to inflate
weights, velocity is more important than predictabilityand weight inflation over-emphasizes predictability.

We do not add weights to bugs as this would be double-counting points. When our delivery contains
bugs, the velocity should go down so we have time to address any systemic quality problems.

Possible Values

We are using the Fibonacci sequence for issue weights. Definitions of each numeric value are associated with the frontend-weight & backend-weight labels. Anything larger than 5 should be broken down whenever possible.

Setting a frontend-weight or backend-weight label on an issue is optional, but ensure you set the Weight property on the issue during refinement.

Examples of when it may be appropriate to set a weight label instead of / as well as setting the issue weight include:

- On newly drafted issues, where we haven't yet fully determined the scope or if both frontend and backend are needed.
- On bugs, where we don't directly assign a weight. The label can help provide guidance on

complexity.

Implementation Plan

A list of the steps and the parts of the code that will need to get updated to implement this feature. The implementation plan should also call out any responsibilities for other team members
or teams. Example.

The goal of the implementation plan is to spur critical analysis of the issue and have the engineer refining the issue
think through what parts of the application will get touched. The implementation plan will also permit other engineers to review the issue and call out any areas of the application that might have dependencies or been overlooked.

Verification Steps

A list of the steps that will need to be followed to verify this feature. The verification steps should also include additional test cases that should be covered. Example.

The purpose of the issue verification procedures is to aid in better understanding the expected change in the application after implementing the issue. Other engineers will be able to evaluate the issue and identify any application components that may have dependencies or have been ignored and that require further testing thanks to the verification steps.

When writing verification steps for a feature or bug fix, it's important to include both positive and negative scenarios. This helps ensure that the feature or fix only works when specific criteria are met and not in every situation. For example, when verifying MR Approval Policies, you should provide a scenario where approval is required when the policy is violated, and another scenario where approval is not needed when the policy is not violated. This approach allows for a more thorough and accurate testing process.

Verification

The issue verification should be done by someone else other than the MR author[^4].

1. All implementation issues should have verification steps in the description. Our implementation issue template conveniently provides this section.
1. When an engineer has merged their work, they should move their issue into the verification status, indicated by the ~workflow:verification label and wait until they receive notification that their work has been deployed on staging via the release issue email.
1. If possible, after the engineer has received the notification and verified their work in staging, they leave a comment summarizing the testing that was complete.
1. After the change is available on .com/production (make sure the MR has the ~workflow:verification label, so it's available with GitLab Next turned off), the engineer should verify again, leave a comment summarizing the testing that was completed, and unassign themselves from the issue. Also provide a link to a project or page, if applicable.
1. Unassigned issues in the ~workflow:verification state are assigned randomly by the triage bot based on the verification policy to an applicable team engineer. This engineer should then additionally verify the issue.

1. Once the issue has been verified in production by both engineers, add the workflow::complete label and close the issue.

[^4]: To minimize cycle time between engineers, it's preferable that the writing engineer verify their work, as they will be able to start working on the issue again immediately if it turns out that the issue has not been sufficiently resolved. Waiting for another engineer to find obvious failures will increase turn around time.

Planning for PTO

We follow the GitLab team members Guide to Time Off and the engineering process to create PTO Coverage issues. PTO Coverage issues are recommended for 3 days or more.

1. Grade 8 team members (EMs, Staff+) should create PTO Coverage issues in the centralized Engineering Division / PTO Coverage project.
1. Other team members should create PTO coverage issues in our Security Risk Management project.

Epic Engineering DRI

As an Epic is ready to move to the refinement stage, the EMs assigns someone as the DRI for each required tech stack. This may happen sooner, during planning breakdown.

As the DRI for an Epic, the engineer is not responsible for executing all the work but they are responsible for:

1. Suggesting the implementation issue breakdown and requesting feedback.
1. Writing the agreed implementation issues.
1. Identifying when further research is required and writing the spike issue(s).
1. Making technical decisions.
1. Providing status updates when requested.
1. Identifying and communicating blockers.
1. Identifying potential security implications and involve a security engineer if necessary
1. Take measurements to reduce the impact of far-reaching work

The DRI may choose to refine and work on the issues they created but they're not expected to deliver the whole Epic on their own.

FAQs

Q: Should discovery issues be refined?

A: Yes. Discovery issues should be refined but some of the steps above may not be relevant. Use good judgement to apply the process above. The purpose of refining a discovery issue is to make sure the scope of the discovery is clear, what the output will be and that the prerequisites for
the discovery are known and completed. Discovery issues can have a habit of dragging out or not creating actionable steps, the refinement process should lock down what needs to be answered in the
discovery process.

Q: If an issue has both frontend and backend work how should I weigh it?

A: Issues that require both frontend and backend work are usually broken into multiple implementation
issues. An exception is when a single engineer agrees to work on both tech stacks.

Q: What's the meaning of the emoji in issues?

A: we use them to communicate certain steps in our process.

· you have reviewed an issue in preparation for Planning Breakdown.
· request to add a specification using Gherkin Keyworks (when life gives you a cucumber, you pickle it).

Footnotes

SECTION: engineering/development/sec/security-risk-management/security-infrastructure/_index.md

Common Links

Slack channels:
Main channel: gsrmsecurityinfrastructure
Stand-up updates: gsrmsecurityinfrastructurestandup

Team Structure

EM: {{< member-by-gitlab "ryaanwells" >}}

{{< member-and-role-by-gitlab "minac" "schmil.monderer" "bala.kumar" "ghavenga" "Quintasan" "srushik" "anarinesingh" >}}

SECTION: engineering/development/sec/security-risk-management/security-insights/_index.md

Customer outcomes we are driving for at GitLab

As a developer it is imperative to know if you are introducing vulnerabilities as you merge into protected branches in addition to the default branch. In FY25, we will allow users to track vulnerabilities across multiple branches. If there is something a developer wants to remediate, but aren't sure where to get started, they can use our features AI to learn more and get a suggestion for a fix.

As a security engineer, you want to know what vulnerabilities to work on first. Over the next year we will be adding key risk metrics so you can quickly triage and mitigate vulnerabilities that have the potential to be exploited.

Leadership wants to make sure their organization is mitigating risks and their security programs are effective. With enhancements to the Security Dashboards, leaders will have a place to get an overview and answer key questions about metrics, trends and vulnerabilities that need to be addressed quickly.

Top Priorities for FY25

Enable users to identify risk and visualize trends - We will be making enhancements to Security Dashboards at the project and group level.

Estimate potential impact and likelihood of vulnerability exploitation - Give users the ability to access risk directly in the vulnerability report through industry known risk scores like CVSS (Common Vulnerability Scoring System) and exploitability probability.

Enable users to track vulnerabilities across multiple branches - Allow users to track vulnerabilities outside the default branch.

Offer guidance for users to get started with vulnerability remediation - leverage the power of AI and security training to help developers understand and remediate vulnerabilities.

Security Insights features are reliable and perform at scale - As we add more group and organization level features, we will be optimizing query performance and move forward with confidence that our database will scale and perform as we grow.

Security Insights Team Structure

The Security Insights group is structured into three focused swimlanes that each approach work in vertical slices: Performance and Optimization, Projects, and AI. This subdivision is to provided bounded focus to each area: enabling us to progress on multiple fronts and reduce planning overhead.

Team Structure

{{% team-by-manager-slug manager="nmccorrison" team="Engineer(.)Security Risk Management:Security Insights" %}}

Common Links

Slack channels:
- Main channel: gsrmsecurityinsights
- Engineering - All SRM groups: ssrmeng

Prioritization

We use our Security Insights Priorities page for 17.x to track what we are doing, and what order to do it in.

Product Workflow

The Security Insights group largely follows GitLab's Product Development Flow.

Additional information can be found on the Planning page.

Milestone Planning

On the second Tuesday of the month the Product Manager kicks off the planning issue. They identify priorities for the milestone and tag engineering managers, and stable counterparts (UX, QA) to review.
By the third Tuesday of the month the Engineering Managers have reviewed the planning issue and agreed on the scope for the milestone.
All issues scheduled for the milestone should have the ~Deliverable label as well as Health Status: On Track at the beginning of the milestone. The milestone field should also be set correctly.
The planning issue is created in this epic for 17.0-17.11.

Project Estimation

Our team follows a multi-phase estimation process. This allows us to have just-in-time information to facilitate predictable roadmap planning.

High Level Estimation

Projects in our priorities roadmap will contain an estimation issue (labeled with ~estimation::needed). These can be found on the estimation issue board.
Estimation issues have several desired outcomes:
Provide high level, of milestone based estimate for the respective capabilities (frontend, backend)
Identify dependencies (other product groups, new technologies, libraries)
Determine outstanding questions and if they block further estimation or will be required before planning breakdown can start.
These outcomes should be added to the respective areas within the epic template.
Add ~estimation:complete label and close the estimation issue when complete.

Tracking Deliverables

Issues that are marked as Deliverables for a milestone serve as the single source of truth for what we aimed to deliver for a given milestone. Throughout the milestone, things may change, become blocked, etc. Ideally, we'd like to keep the Planning Issue unchanged after the milestone starts.
Something is considered delivered if it is either a. merged into production in time for the release date, b. completed before the next milestone start, or c. the feature flag enabling the feature is turned on. It is important to keep track of the milestone of the deliverable; we encourage self-managed customers to turn on feature flags so they can try different features. Ensuring the milestone is correct, allows someone to tell if that change is available in a specific release.

Weekly async issue updates

At the end of every week, each engineer is expected to provide a quick async issue update by commenting on their assigned issues using the following template:

markdown
 Async issue update

 Current status:
<!--- Please provide a quick summary of the current status (one sentence) -->

 Shipping this milestone: <!-- Not confident | Slightly confident | Very confident -->

 Scope reduction opportunities: <!-- No | Yes, ... -->

/healthstatus <!-- ontrack | needsattention | atrisk -->
/label <!-- ~"workflow::blocked" | ~"workflow::refinement" | ~"workflow::ready for development" | ~"workflow::In dev" | ~"workflow::In review" | ~"workflow::verification" -->

<!-- Please apply a :triangularflagonpost: emoji to this comment. Fore more information see https://gitlab.com/jayswain/automated-reporting -->

We do this to encourage our team to be more async in collaboration and to allow the community and other team members to know the progress of issues that we are actively working on. This also enables us to automatically collate updates across swimlanes, removing some manual process.

 Indicating Status and Raising Risk

Our teams use the Health Status feature within issues to indicate the likelihood of completion within the milestone. We assign On Track at the beginning of a milestone to a small number of issues where we have high confidence in delivery during that milestone. If there is concern with marking something as initially on track, then we should discuss why.

Raising risk early is important. The more time we have, the more options we have. For example, issues that have not gone into review by the first Monday of the month may not have enough time to get merged. These should be considered Needs Attention or At Risk depending on their complexity and other factors.

Follow these steps when raising or downgrading risk:

1. Update the Health Status in the issue:
    1. On Track - high confidence - there is no indication the work won't get merged by the second Tuesday of the month.
    1. Needs Attention - medium confidence - the issue is blocked or has other factors that need to be discussed.
    1. At Risk - low confidence - the issue is in jeopardy of missing the merge cutoff on the second Tuesday of the month.
1. Add a comment about why the risk has increased or decreased. Copy the Engineering Manager and Project Manager for awareness.

Note that an issue probably shouldn't go directly from On Track to At Risk. That pattern indicates we have missed an opportunity to discuss earlier. Consider the progression: On Track -> Needs Attention -> At Risk.

Support rotation

On top of our development roadmap, engineering teams need to perform tasks related to support and triage. Our team nominates an individual person to reserve capacity for these tasks. The rota is here (internal link) This is to avoid excessive context-switching and better distribute the workload. It is important we defend our focus within the team to support the delivery of our commitments.

If you are not the nominated person in a given week then:

1. You are not expected to triage and investigate by default. Use your best judgement here (e.g. critical issues still take priority, no change in expectations here).
1. You should redirect the question to the nominated person (e.g. if it comes in a DM in Slack, redirect it to our public channel).

Please keep track of the actions you're doing during your rotation and add notes in the corresponding issue (e.g. copying tools command executed locally, sharing relevant changes to projects and processes, etc.)

Triage expectations

Triage does not immediately guarantee a change to currently-planned work in a milestone. Triage is the process of determining impact and priority so we can justify changes to scope and milestone commitments.

Refine the request for help tickets: do we have reproduction steps, does this relate to other scoped or planned work, is this a bug or feature request or an acceptable limitation of the system.
Outcomes could be: updates to our documentation or Handbook pages, validated reproduction of bugs and then creating issues from this.
Directly answering support questions.
Engaging with Product to agree on priority and scheduling of any work required. Work with Product to define severity and whether to interrupt the rest of the development team.

When dealing with Slack interactions you are expected to use the following reactions:

:eyes: - I am actively looking at this
:whitecheckmark: (or a variant) - This is resolved

Responsibilities - Support

1. Monitor slack channels for questions, support requests, and alerts. The person assigned to the reaction rotation is expected to handle them primarily.
If a support engineer requests assistance via Slack and it requires investigation or debugging, they should be directed to raise an issue in a dedicated project.

gsrmsecurityinsights
ssrm
sec-section

We utilize a standardized Request for Help process to request formal assistance from our group . This helps with visibility, tracking and review. Please submit a new Request for Help for Security Insights using this template.

MR Reviews

We follow these guidelines when submitting MRs for review when the change is within the Security Insights domain:

1. Aim to request at least one of the reviews from someone outside our group. This helps avoid a code knowledge silo.
1. For time-critical reviews, consider using internal reviewers and maintainers.
1. The initial review should be performed by a member of the team. This helps the team by:
   Faster reviews, as the reviewer is already familiar with the domain.
   Additional awareness of changes taking place within the domain.
   Identifying changes that don't align with what is happening with the domain.
   Providing additional confidence from a domain expert to the external maintainer reviewer that the change behaves as expected.
1. GraphQL merge requests should be reviewed by a frontend engineer as soon as possible. This helps to validate the interface, and allows changes to be made before tests are written.

Issue Boards

Security Insights Milestone Board
Primary board showing the stage of currently planned issues.

Security Insights "Who's working on what" board
Shows issues assigned to engineers on our team.

These boards show current status of issues.

Quality

Quality and E2E Specs

Workflow of E2E runs on Staging and Production

We run scheduled E2E tests on both staging and production environments every 4 hours. These tests help ensure that recent deployments haven·t introduced regressions.

We can monitor test results in the following Slack channels:

e2e-run-staging
e2e-run-production

Running and Fixing E2E specs

Prerequisites

Ensure the following before running tests:

gdk is up and running
Runner is up and running
Set GITLABSIMULATESAAS to 0 inside your env.runit in the gitlab-development-kit directory:

shell
export GITLABSIMULATESAAS=0

Ensure EE License is set as an environment variable.

Running QA Tests

Use the following command to run tests locally against your GDK instance:

Running against your gdk

With a feature flag enabled:

shell
WEBDRIVERHEADLESS=false bundle exec bin/qa Test::Instance::All http://gdk.test:3000/<filename/path> --enable-feature <featureflagname>

You can also run a specific RSpec line using <filename>:<linenumber> to target the surrounding example block. See RSpec best practices for more details.

With a feature flag disabled:

shell
WEBDRIVERHEADLESS=false bundle exec bin/qa Test::Instance::All http://gdk.test:3000/<filename/path> --disable-feature <featureflagname>

Without a feature flag:

shell
WEBDRIVERHEADLESS=false GITLABADMINPASSWORD="rootpassword" GITLABQAADMINACCESSTOKEN="apitokenfromgdk" GITLABPASSWORD="rootpassword" QALOGLEVE
QAGITLABURL=http://gdk.test:3000 bundle exec rspec <filename/path>

Running against staging

shell
GITLABQAUSERAGENT=<USERAGENT> GITLABADMINUSERNAME=<ADMINUSERNAME> GITLABADMINPASSWORD=<ADMINPASSWORD> GITLABUSERNAME=<USERNAME> GITLABQAACCESSTOKEN=<ACCESSTOKEN> GITLABPASSWOR
QADEBUG=true WEBDRIVERHEADLESS=true bundle exec bin/qa Test::Instance::All https://staging.gitlab.com <filename/path>

The credentials are to be found in 1Password.

Local testing of licensed features

When a feature needs to check the current license tier, it's important to make sure this also works on GitLab.com.

To emulate this locally, follow these steps:

1. Export an environment variable[^1]:

```
shell
export GITLABSIMULATESAAS=1
```

1. Within the same shell session, run:

```
shell
gdk restart
```

1. Navigate to Admin > Settings > General > "Account and limit", and enable "Allow use of licensed EE features".

See the related handbook entry for more details.

Troubleshooting common errors and fixes

Error: QA::Resource::Sandbox Fabrication Failed
Error Message:

```
plaintext
Fabrication of QA::Resource::Sandbox using the API failed (400) with { "message": "Failed to save group {:visibilitylevel=[\"public has been restricted by your GitLab administrator\"]}"
}
```

Solution:
Navigate to GDK Admin Area · General
Under Restricted Visibility Levels, ensure none of the checkboxes are selected.

Error: API Client Validation Failed
Error message:

```
plaintext
An error occurred in a before(:suite) hook.
Failure/Error: raise InvalidTokenError, "API client validation failed! Code: {resp.code}, Err: '{resp.body}'"
```

Solution:

Ensure your user verification is complete before running a pipeline.
Check if your API token is valid.

Error: Namespace is Not Valid

Error message:

plaintext
QA::Resource::Errors::ResourceFabricationFailedError:
Fabrication of QA::Resource::Project using the API failed (400) with { "message": { "namespace": ["is not valid"] } }.

Solution:

Reset your GDK by running:

shell
gdk data-reset

Running E2E specs in the MR pipeline

We encourage running the e2e: test-on-omnibus downstream E2E job in merge requests at least once and reviewing the results when there are changes in:

- GraphQL (API response, query parameters, schema, etc.)
- Gemfile (version changes, adding/removing gems)
- Database schema/query changes
- Any frontend changes that directly impact the vulnerability report page, MR security widget, pipeline security tab, security policies, configuration, or license compliance page.

Running Govern E2E specs locally against GDK

Standalone E2E specs can be run against your local GDK instance.

E2E tests with feature flags

E2E tests should pass with a feature flag enabled before it is enabled on Staging or on GitLab.com.
Therefore, it's important to confirm this when introducing a new feature flag. Adding or editing a feature flag definition file starts two e2e:test-on-omnibus jobs (one with the feature flag turned on and another where it's turned off).

For a thorough explanation of the end-to-end testing process when working with feature flags, please consult the official documentation on the Testing feature flags with end-to-end tests page.

Notes and Resources on QA Testing

For any questions, reach out to sdeveloperexperience.

Resources

Testing Code in Merge Requests
Running Govern E2E Specs Locally Against GDK
Automatic test execution when a feature flag definition changes
End-to-end test pipelines
GitLab team member's guide to using official build infrastructure

Monitoring

Stage Group dashboard on Grafana
Largest Contentful Paint (LCP) for our web pages.

Contributing

Local testing of licensed features

When a feature needs to check the current license tier, it's important to make sure this also works on GitLab.com.

To emulate this locally, follow these steps:

1. Export an environment variable: export GITLABSIMULATESAAS=1
   There are many ways to pass an environment variable to your local GitLab instance. For example, you can create a env.runit file in the root of your GDK with the above snippet.
1. Within the same shell session run gdk restart
1. Admin > Settings > General > "Account and limit", enable "Allow use of licensed EE features"

See the related handbook entry for more details.

Cross-stack collaboration

We encourage frontend engineers to contribute to the backend and vice versa. In such cases we should work closely with a domain expert from within our group
and also keep the initial review internal.

This will help ensure that the changes follow best practice, are well tested, have no unintended side effects, and help the team be across any changes that go into the Security Insights codebase.

Community Contributions

The Security Insights group welcomes community contributions. Any community contribution should get prompt feedback from one of the Security Insights engineers. All engineers on the team are responsible for working with community contributions. If a team member does not have time to review a community contribution, please tag the Engineering Manager, so that they can assign the community contribution to another team member.

If a team member creates an issue or finds an issue where we would be open to a community contribution, it should be labeled with ~"Seeking community contributions". If the contributor needs an EE license, we can point towards the Contributing to the GitLab Enterprise Edition (EE) section on the Community contributors workflows page.

Group discussion

We hold group discussions every other week. We alternate between a milestone kickoff and general discussion format. Everyone is invited to attend, and it's a great forum to ask questions about Vulnerability Management, customer queries, our road map, and what the Security Insights team might be thinking about. You can find the meetings on the Security Insights calendar; take a look at the agenda (internal link). We hope to see you there!

Metrics

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="security insights" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="security insights" >}}
{{< /tableau >}}

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssuesDetails" >}}
  {{< tableau/filters "GROUPNAME"="security insights" >}}
{{< /tableau >}}

SECTION: engineering/development/sec/security-risk-management/security-insights/developer_setup.md

Requirements

Set up GDK

To fully run Vulnerability Management on your local machine, you must have set up the GDK.

Set up runner

To display the Vulnerability Reports, you need to set up the runner. Follow these steps:

1. Navigate to http://gdk.test:3000/gitlab-org/security-reports .
1. On the left sidebar, click on the Search or go to... button and select Admin Area.
1. In the Admin Area, on the left sidebar, select CI/CD > Runners.
1. Select New instance runner > Run untagged jobs > Create Runner.

1. Choose your Operating system and follow the instructions of Step 1.
1. Ensure that Docker is running on your machine.
1. Open your terminal, run gdk start. Once gdk is running, run the command gitlab-runner run.
1. Return to your browser, and click on View runners. Your runner should be shown in the list of runners, and show as Online.
1. Navigate back to the Security Reports project at http://gdk.test:3000/gitlab-org/security-reports.
1. On the left sidebar click on Build > Pipelines. The pipeline should now be active.

For additional details or troubleshooting, consult the official runner setup guide.

Ensure EE license

To display Vulnerability Reports and the Vulnerability Management tool in GitLab, you need an Enterprise Edition (EE) license. This license enables features exclusive to the EE tier. To generate an EE development license, follow these steps:

1. Request an EE developer license. Follow the steps in the handbook.
1. Add the EE license to your local environment. Follow the steps in the handbook under Add license in the Admin area.

Resources and examples

Repositories

To easily populate vulnerabilities, we recommend the Security Reports project. To add it to your local GDK environment:

1. Go to http://gdk.test:3000/ in your browser.
1. Click on New Project > Import Project > Repository by URL.
1. In the Git repository URL field, enter https://gitlab.com/gitlab-examples/security/security-reports.git.
1. Under Project URL, add a namespace, (for example, gitlab-org).
1. For Project slug enter security-reports.
1. Click Create project.

SECTION: engineering/development/sec/security-risk-management/security-insights/ve_vr_setup.md

Setup Guide for Vulnerability Explanation and Resolution

Several setup steps are necessary in order to test and develop Vulnerability Explanation (VE) and Vulnerability Resolution (VR) features locally. This guide contains instructions for setting up and configuring the necessary components.

Setup Runner

To generate vulnerability reports you will need to run a CI pipeline. Follow the instructions

below to install and configure the GitLab Runner.

https://gitlab.com/gitlab-org/gitlab-development-kit/blob/main/doc/howto/runner.md

Optionally, you can install Docker with Colima. The docs can be found here or you can follow the steps in this snippet.

Setup Vulnerability Report

The VE and VR features are designed to work on SAST vulnerabilities. Clone one or more of the following projects to use for local testing:

https://staging.gitlab.com/govern-team-test/oxeye-rulez
https://gitlab.com/gitlab-org/security-products/tests/webgoat.net
https://gitlab.com/gitlab-examples/security/security-reports
https://gitlab.com/gitlab-org/govern/threat-insights-demos/verification-projects/cwe-samples

Run the pipeline on the main or master branch for any of the sample projects to generate the vulnerability report. Build > Pipelines > Run Pipeline

Once the pipeline is finished, the Vulnerability Report can be viewed by going to Secure > Vulnerability Report > Any SAST finding

Examples:

Vulnerability Report: https://gitlab.com/gitlab-org/security-products/tests/webgoat.net/-/security/vulnerabilityreport
SAST finding: https://gitlab.com/gitlab-org/security-products/tests/webgoat.net/-/security/vulnerabilities/105323245

Setup AI

Follow this instructions here to configure your GDK access to AI features.

For GitLab Team members only:

An EE license is required, follow the steps here to request one.
Anthropic access is required. Create an access request if necessary (example).

Duo Access

Once you have AI set up locally, you will need to enable Duo features. Follow the steps below to ensure you have everything correctly configured.

Follow this instructions here to setup and run GDK.

Usage

With the configuration in place, you should expect to see the Explain with AI button for any SAST vulnerability. For example: https://gitlab.com/gitlab-org/security-

products/tests/webgoat.net/-/security/vulnerabilities/105323245

You should expect to see the 'Resolve with AI` button for any SAST vulnerability in the high confidence CWE list. For example: https://gitlab.com/gitlab-org/security-products/tests/webgoat.net/-/security/vulnerabilities/114941072

If you need assistance, please reach out in ggovernthreatinsightsengai

---
SECTION: engineering/development/sec/security-risk-management/security-insights/ve_vr_troubleshooting.md

Troubleshooting Resource Guide for VE and VR

When working with Vulnerability Resolution and Vulnerability Explanation, you might run into an error. Most commons
problems are documented in this section.
If you find an undocumented issue, you should document it in this section after
you find a solution.

If you need help developing or testing locally, please see the setup guide.

For availability of these features please first check the prerequisites listed here:
vulnerability explanation and vulnerability resolution.

Also check: VR troubleshooting guide.

| Problem | Solution
|
|---|---

---|
Duo / VR features aren't available	The group/project may not have assigned Duo Seats. Follow the Duo subscription add-ons instructions.
Upstream errors such as "The upstream AI provider request timed out without responding"	This may indicate an issue with our third-party AI. This could be Anthropic outage - check status.
Specific recurring errors like "an unexpected error has occurred"	This may indicate an issue with the creation of the diff patch or MR. Refer to Error handling code
False positive errors	We handle empty responses and empty <fixedcode> as false positives. Documentation, Response modifier code
If you see that the VR button is disabled, that means that the CWE is not part of the supported list at this time.	Feature coverage restriction: VR is available for a set of CWEs, check SSOT documentaton and spreadsheet.
Query custom errors in Elastic	Check this dashboard for further investigation.

CWE Support

Vulnerability Explaination

Vulnerability Explaination is enabled for all SAST vulnerabilities.

Vulnerability Resolution

Vulnerability Resolution is enabled for SAST vulnerabilities, only for a specific set of CWEs documented at Supported vulnerabilities for Vulnerability Resolution.

We determine whether a vulnerability supports Vulnerability Resolution based on its CWE identifier. This support is tracked using two mechanisms:

1. Database field on vulnerability records hasvulnerabilityresolution

   The database field is populated and backfilled during ingestion, meaning any successful pipeline run on the default branch after a CWE list update will ensure it contains the latest values.

   This field is used, for example, in:
   - The Vulnerability Report (for filtering and display)
   - Vulnerability Details (e.g., availability of "Resolve with AI")

   > Attention: Background migrations aren·t strictly required to backfill this value, but they are currently an established part of our workflow (see example migration). Any changes to this process must be clearly documented.
1. Hardcoded list
   - Used for pipeline findings (e.g., in merge requests), meaning they haven·t been fully ingested as vulnerability records yet and the hasvulnerabilityresolution field in the database remains unset.

> Note: Unsupported CWEs may be tested by enabling the ignoresupportedcwelistcheck feature flag at the project level (MR)

Dashboard to see logs

1. Production log dashboard - shows request/response/error as well as p50/p90/p99 for the timings of the duo request
1. Staging log dashboard

Monitoring VR alerts

1. Elastic watcher
1. Slack channel to see alerts: gsrmsecurityinsightsaierroralerts
1. Elastic logs used in watcher: https://log.gprd.gitlab.net/app/r/s/foNLr
1. Error watcher in IaC repo: https://gitlab.com/gitlab-com/runbooks/-/blob/master/elastic/managed-objects/loggprd/watches/testgsrmsecurityinsightsaierrorwatcher.jsonnet
1. Alert threshold for this watcher is 5 errors over last 90 minutes. If needed the watcher can be deactivated from this page. The threshold value can be changed from the edit page.

Resources

1. Documentation
   - Vulnerability Resolution
   - Vulnerability Resolution in the MR
1. LLM Prompts for VE and RV

SECTION: engineering/development/sec/security-risk-management/security-insights/vulnerability_archival_setup.md

Vulnerability Archive Generation Guide

This guide outlines the steps to generate a vulnerability archive in your local environment.

Requirements

Existing Vulnerabilities

To generate vulnerabilities locally, follow the instructions in the Developer Setup Guide.

Generating an Archive

The vulnerability archive feature is gated behind the vulnerabilityarchive feature flag. Follow these steps to enable and use it:

Enable the Feature Flag

Open your terminal and run the following command to enable the feature in the Rails console:\

bash
```
echo "Feature.enable(:vulnerabilityarchival)" | rails c
```

Locate the Project ID

Navigate to your project·s homepage. Click the vertical dot menu (·) and expand it to reveal the project ID. Copy this ID for the next step.

Open the Rails Console

In your terminal, start the Rails console by running:

bash
```
rails c
```

Generate the Archive

Execute the following command in the Rails console, replacing <projectid> with the ID you copied:

ruby
Vulnerabilities::Archival::ArchiveService.execute(Project.find(<projectid>), Date.today + 1.day)

This command will:

- Delete all vulnerabilities up to the current date.
- Create a new archive containing those vulnerabilities.

View the Archive

Once the archive is generated, you can view it by navigating to:

Secure > Security Configuration > Vulnerability Management from the left navigation bar.

SECTION: engineering/development/sec/software-supply-chain-security/_index.md

The Software Supply Chain Security sub-department teams are the engineering teams in the Software Supply Chain Security Stage of the product.

Vision

To support GitLab's product vision through alignment with the Software Supply Chain Security stage product direction.

Groups

- Authentication
- Authorization
- Compliance
- Pipeline Security

Priorities

Group priorities are reviewed collaboratively with product counterparts and published on the Software Supply Chain Security direction pages

- Authentication
- Authorization
- Compliance
- Pipeline Security

Product Documentation Links

- Authentication and Authorization
- Compliance Center
- Security glossary
- Pipeline Security

All Team Members

Authentication

{{% team-by-manager-slug manager="adil.farrukh" team="Engineer(.)Software Supply Chain Security:Authentication" %}}

Authorization

{{% team-by-manager-slug manager="jayswain" team="Engineer(.)Software Supply Chain Security:Authorization" %}}

Compliance

{{% team-by-manager-slug manager="nrosandich" team="Engineer(.)Software Supply Chain Security:Compliance" %}}

Pipeline Security

{{% member-and-role-by-gitlab "fcatteau" "ahuntsman" "cipherboy-gitlab" "dbiryukov" "iamricecake" "jmallissery" "mgandres" "srajadas" "sroque-worcel" %}}

Stable Counterparts

The following members of other functional teams are our stable counterparts:

{{% engineering/stable-counterparts role="Software Supply Chain Security" other-manager-roles="Engineering Manager(.)Software Supply Chain Security:(.)|Director of Engineering(.)Software Supply Chain Security" %}}

Software Supply Chain Security staff meeting

The Software Supply Chain Security stage engineering department leaders meet weekly to discuss stage and group topics in the Software Supply Chain Security staff meeting. This meeting is open to all team members and is published on the Software Supply Chain Security stage calendar.

Meetings have an agenda and are async-first, where the aim is to resolve discussions async and leave time in the meeting to deep dive into topics that require more discussion.

We use the Software Supply Chain Security Sub-department Board to better organize our discussions.

Weekly updates

The Software Supply Chain Security development teams provide weekly status updates using an

issue template and CI scheduled job.
As priorities change, engineering managers update the template to include areas of interest such as priorities, opportunities, risks, and security and availability concerns. The updates are GitLab internal.

Quarterly review updates

Every quarter, an engineering manager for each group in the Software Supply Chain Security Sub-department prepares the quarterly review update using the issue template and records approximately 5 minutes to summarize the last quarter from the engineering perspective and present a high-level plan for the group for the next one to respond to quarterly Product strategy and explain our goals for next quarter.

We aim to foster collaboration and communication between engineering managers in the Software Supply Chain Security Sub-department, align groups on product priorities for the next quarter, and celebrate our successes together.

Quarterly review update template can be found here).

OKR planning

Our OKRs are a mixture of top down, aligned with Company-wide, Product, or Engineering Division OKRs, and bottom up engineering-led initiatives driven by our team members in Software Supply Chain Security. Any team member can propose an OKR for Software Supply Chain Security by creating a proposal issue in our internal OKR project. The issue can be used to collaborate and discuss the proposal. When we are ready to commit, we can create or align to an existing Objective, and add specific key results to track through the quarter.

Labels:

- Sub-Department::software supply chain security - for top-level sub-department Objectives.
- devops::software supply chain security - for Objectives and key results for the stage, and stage groups
- group:: - for Objectives and key results for a specific group

Each Objective and Key Result should have an assignee who is DRI for providing status updates throughout the quarter. Regular updates are preferred. At a minimum these should be updated

- By end of day, the second Friday of every month
- Ay the end of the quarter

OKRs can be changed or closed during the quarter if they are completed, or as our goals change. This ensures we are focusing on areas that are revelant to our current and future priorities.

PTO

We follow the Engineering process for taking time off and GitLab team members Guide to Time Off.

Engineering Leadership - PTO or unavailable

Team members should contact any Software Supply Chain Security Engineering Manager by mentioning in sdsscsengineering or sscs-development-people-leaders if they need management support for a problem that arises, such as a production incident or feature change lock, when their direct manager is not available. The Software Supply Chain Security manager can provide guidance and coordination to ensure that the team member receives the appropriate help.

Some people management tasks, including Workday and Navan Expense, may require for escalation or delegation.

Skills

Because we have a wide range of domains to cover, it requires a lot of different expertise and skills:

Technology skills	Areas of interest
Ruby on Rails	Backend development
Go	Backend development
Vue, Vuex	Frontend development
GraphQL	Various
SQL (PostgreSQL)	Various
Docker/Kubernetes	Threat Detection

Everyone can contribute

At GitLab our goal is that everyone can contribute. This applies to GitLab team members and the wider community through community contributions. We welcome contributions to any and all features, but recognize that first time contributors may prefer to start with smaller features.
To support this we maintain a list of quick wins that may be more suitable for first time contributors, and contributors new to the domains in Software Supply Chain Security.

- Quick wins

If the contributor needs an EE license, we can point towards the Contributing to the GitLab Enterprise Edition (EE) section on the Community contributors workflows page.

Testing

During the planning phase of a milestone, the EM for each group will create a new issue using the template in epic, for any major new features and tag Software Engineer in Test from Software Supply Chain Security. SETs from Test Engineering and EMs can periodically review/discuss the list of open issues, and add appropriate priority labels.

The intent of shifting left and testing at the right level is that teams are responsible for testing and to have engineers doing the feature coverage reviews and adding specs or E2E test as needed. The reason for including the SET is to give oversight across the groups and provide guidance/support. If the SET has capacity then they can contribute as needed, using the priority labels, but this is not the expectation.

Links and resources

{{% include "includes/engineering/softwaresupplychainsecurity-shared-links.md" %}}

Technical Documentation Links

- End-to-end tests

SECTION: engineering/development/sec/software-supply-chain-security/anti-abuse.md

Vision

Our goal is to provide Insider Threat features for your applications as well as GitLab itself.
We will help proactively identify malicious activity, accidental risk, compromised user accounts or infrastructure components, anomalous use of the GitLab platform, and various high-risk behaviors where actionable remediation steps are possible.

Direction

- Instance Resiliency
- Insider threat

Planning

Our planning issues are the SSOT of what we're working on now, and what we're working on next. We also have an issue board to view these from a workflow perspective. To maintain the issue lists leadership (EM+PM) will keep the lists triaged.

Workflow

We follow the same workflow pattern as our friends in Govern::Authorization.

Iteration

When planning how to construct our MVC, we need to be aware of the tradeoffs of slicing MR's vertically vs horizontally. Reducing scope for each iteration is encouraged.

As requirements can shift, and complexity can increase when uncovering challenging areas in the codebase, we strive to keep issue requirements updated for clarity.

We follow the iteration process outlined by the Engineering function.

Weekly async issue updates

We use the same weekly async issue template as our friends in Govern::Authorization.

Group members

Anti-abuse group can be @ mentioned on GitLab with @gitlab-org/modelops/anti-abuse.

The following people are permanent members of the group:

{{< team-by-manager-slug manager="jayswain" team="(?i)Engineer(.)Govern:Anti-abuse" >}}

Team Meetings

Our group holds synchronous meetings to gain additional clarity and alignment on our async discussions. We aspire to record all of our meetings as our team members are spread across several time zones and often cannot attend at the scheduled time.

We have a weekly team sync meeting with rotating AMER and AMER/APAC friendly time slots: Tues 18:30 UTC and Weds 00:00 UTC.

Collaboration

You are encouraged to work as closely as needed with our stable counterparts.

Other teams that we might collaborate with include but are not limited to:

- Govern:Authentication and Authorization
- Growth:Acquisition and Activation
- Fulfillment:Fulfillment Platform

Here are some examples of when to engage with your counterpart:

- Seeking a Govern:Authentication and Authorization review when making a significant change to the registration flow
- Seeking a Fulfillment review when making a change involving Zuora
- Discussing in fsignupregistration (Slack, GitLab internal) when making a change that affects how our users signup or login

Abuse Maintenance

The Anti-abuse team works closely with Trust and Safety to mitigate abuse on our platform. It's not uncommon for Trust and safety to request features or maintenance from our team to assist their effort of mitigating abuse. Prioritized requests are organized in our Abuse Maintenance epic.

Pipeline Validation Service responsibility

PVS is an internal service that belongs to the Anti-abuse team. It's a combination of heuristic-based (text matching, etc) and behavior-based rules (duplicate builds, etc). The Trust and Safety team leverages this service the most, and acts as the customer for feature requests.

Heuristic rules

Due to the nature of cryptomining attacks, heuristics are going to change quickly and need to be implemented rapidly. Accordingly, T&S is invited to submit MR's to PVS that are heuristic based, or alternatively request these changes from the Anti-abuse team.

Behavior rules

Behavior rules are more slow to change and potentially cast a much wider net (vs a very targeted heuristic rule). Changes to behavior rules are expected to come from T&S, and implemented by the Anti-abuse team.

Severity and Priority

Severity and priority will be added on all issues/merge requests created by T&S so that Anti-abuse can act on them accordingly.

Priority will be based on impact and likelihood of the attacker returning.

Iteration

Anti-abuse will periodically review the accuracy of PVS alerts to see where there are opportunities to reduce the False Positive rate, without impacting the true positives, and Trust and Safety will help provide the required information to do this.

Links and resources {links}

- Our Slack channels
  - Govern:Authorization ggovernanti-abuse

Metrics

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="anti abuse" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="anti abuse" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="anti abuse" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="anti abuse" >}}
{{< /tableau >}}

SECTION: engineering/development/sec/software-supply-chain-security/authentication.md

SSCS:Authentication{welcome}

Mission

Our mission is to empower GitLab system administrators with the toolkit they need to create their desired balance of security and accessibility for their GitLab experience. Authentication is the first impression any new customer has when they configure their shiny new GitLab instance, and we aim to make it as seamless as possible: from that moment of first logging in, to onboarding users, to managing the basic security rules for their instance in a secure, flexible and scalable manner.

Top Priorities for FY25

Our detailed priority list can be found at the direction page however on a higher level the focus would be on:

1. GCP integration
2. Cells readiness
3. Service accounts MVC features
4. Passwordless authentication
5. Enterprise user admin controls and policies management
6. Bringing Credentials inventory to GitLab.com
7. Reducing the number of flakey tests, older FFs, S3 bugs and manual handing of support/CSM questions.
8. Group SCIM Sync support
9. Service accounts UI and enhanced capabilities
10. Token management enhancements
11. Credential Manager enhancements

Customer Outcomes we are driving for GitLab

As a result of the above roadmap items,